

Managing Project Source Code Development Efficiently with Git

The first part of this article introduced Git, its workflow, installation and basic commands. This second part elaborates the collaboration aspects of Git, including branching and merging, which help with parallel, non-linear development in projects and avoid disruption in the main line of development. Working with remote repositories is also covered.



Branching in Git is analogous to branches on a tree. A Git branch is an independent line of development emerging from the main development line, just like branches coming out of the trunk of a tree. This feature at its core enables the collaboration of a team on a single project by letting developers work on feature developments, bug fixes or hot fixes without disturbing the main development line on their own branches, and merging them whenever necessary.

In most version control systems (VCS), branching is an expensive operation with respect to both time and space. Git branches are extremely lightweight, making operations like creating branches and switching back and forth from one to the other very quick. This feature sets Git apart from the other VCS.

Creating and using branches

`git branch <branch-name>` is used to create a new branch. It creates a new pointer from the current branch. When the branch is put to use, it does affect the current branch whenever any changes or commits are made in the repository. `$git checkout <branch-name>` is used to make the HEAD point to the `<branch-name>`. In other words, this updates the files in the repository to match the version stored in the branch, and tells Git to record any changes from now on to the branch.

Both these commands can be combined in a single command like `git checkout -b <branch-name>`. Let's create a feature branch and record some changes in it, as explained in Figure 1. We will continue working on the same repository that was created in the first part of this article carried in the August 2020 issue of *OSFY*.

